INTERPROCESS-COMMUNICATION

IPC와 공유 메모리 동기화

연관 기출 · 136-4-2

한눈에 이해

프로세스는 서로 메시지를 보내거나 같은 메모리를 사용해 협력합니다. 같은 메모리를 쓴다면 언제 읽고 쓸지 정하는 규칙이 필요합니다.

정의와 핵심 원리

프로세스 간 통신(IPC)은 운영체제가 제공하는 데이터·이벤트·상태 교환 방식입니다. 서로 분리된 프로세스가 정보를 주고받으며 작업을 나눌 수 있습니다.

• 파이프는 바이트를 연속해서 전달합니다. 익명 파이프는 주로 서로 관련된 프로세스 사이에서 쓰고, 이름 있는 파이프(FIFO)는 경로를 열어 접속합니다. 메시지 큐는 메시지 경계를 유지하고 우선순위를 두거나 비동기로 주고받기 쉽습니다. 복사 비용과 큐 용량은 따로 관리합니다.

• 공유 메모리는 여러 프로세스가 같은 영역을 각자의 주소 공간에 매핑하는 방식입니다. 대량의 데이터를 복사하는 비용을 줄일 수 있습니다. POSIX에서는 shm_open → ftruncate → mmap 순서로 준비하고 사용이 끝나면 정리합니다. 읽기·쓰기의 원자성, 가시성, 순서는 세마포어나 프로세스 공유 뮤텍스 같은 동기화 수단으로 보장해야 합니다.

• 소켓은 로컬 UNIX 도메인이나 네트워크를 통해 데이터를 주고받습니다. 시그널은 이벤트를 알리는 데 적합하며 복잡한 데이터를 전달하기는 어렵습니다. 전달량, 지연, 메시지 경계, 통신 주체의 관계, 접근 권한과 오류 복구를 기준으로 기법을 고릅니다.

구조·비교·판단

구분	핵심 원리	적용·한계
파이프·큐	커널이 데이터를 전달하고 버퍼 관리	메시지 경계·흐름 제한·용량
공유 메모리	여러 프로세스에 같은 페이지 매핑	동기화·메모리 수명 관리
소켓	로컬·원격 통신 끝점	직렬화·접속·부분 전송

적용 예제

유한 버퍼 생산자·소비자

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 버퍼는 4칸입니다. 세마포어 초깃값은 empty = 4, full = 0, mutex = 1입니다. P는 자리를 하나 확보할 때까지 기다리는 연산, V는 자리를 하나 반환하는 연산입니다.

1. 생산자는 P(empty) → P(mutex) → 쓰기 → V(mutex) → V(full) 순서로 처리합니다.

2. 소비자는 P(full) → P(mutex) → 읽기 → V(mutex) → V(empty) 순서로 처리합니다.

3. 버퍼가 비면 소비자는 full을, 가득 차면 생산자는 empty를 기다립니다.

4. 버퍼가 가득 찼을 때 생산자가 mutex를 먼저 잡고 empty를 기다리면, 소비자가 mutex를 얻지 못해 버퍼를 비울 수 없습니다.

결과·해석: empty와 full은 쓸 자리와 읽을 데이터의 수를 관리하고, mutex는 버퍼에 한 번에 한 프로세스만 접근하게 합니다.

전제·주의: 개념을 설명하는 의사코드입니다. 작업 취소·프로세스 종료·복구 정책은 운영 요구에 맞춰 보완합니다.

답안 골격과 혼동 방지

IPC의 필요성 → 기법별 구조 → 통신과 동기화의 역할 → P/V 버퍼 예제 → 성능과 복구 판단

- 공유 메모리와 네트워크 공유 파일을 구분합니다.
- 메시지 큐만으로 애플리케이션 트랜잭션을 보장할 수 없습니다.

내용을 가리고 회상

1. 같은 메모리에 접근할 때 동기화가 필요한 이유는 무엇입니까?

2. 파이프와 메시지 큐는 메시지 경계를 어떻게 다룹니까?

3. 큐가 가득 차도 생산 속도를 줄이지 않으면 어떤 문제가 생깁니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- shm_overview(7) - Linux man-pages project; 상시 문서; 확인 2026-11-05.
- Lock types and their rules - Linux Kernel documentation; 상시 문서; 확인 2026-11-05.